Trabalho de Compiladores

Gabriel Otavio da Cunha Cota[1], **Lucas Alonso Correia de Oliveira**[1]

[1]Bacharelado em Ciência da Computação
Universidade Federal Fluminense (UFF)
Niterói – RJ – Brasil

gabrielotavio@id.uff.br, lcorreia@id.uff.br

1. A construção do Compilador da linguagem C-

Abstract. *The text is a detailed report about how is build a compiler. Since lexical analyzer til machiene language codification and it's optimization. In the moment, we built the lexical analyzer of C- language from Louden's book.*

Resumo. *O texto a seguir é um relatório detalhado sobre como o compilador foi construído. Desde o analisador léxico até a codificação na linguagem de máquina envolvendo a sua otimização. Nesse momento inicial, só será abordada a construção do analisador léxico da linguagem C- em [Louden 2004].*

1.1. Introdução

Segundo [Louden 2004] o compilador é primeiramente um software que tem como tarefa fazer a tradução da linguagem da linguagem fonte (geralmente de alto nível) pra linguagem alvo, que é o código em máquina. Todo o processo de tradução que leva a transformação em linguagem de máquina se dá conforme demonstrado na 1.

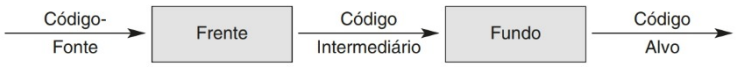


Figura 1. Esquema do funcionamento de um compilador.

O Bloco de Frente ou front-end corresponde ao processo de análise que corresponde a análise léxica, sintática e semântica, já o Fundo ou Back-End gerará através do código intermediário gerado pelo front-end, transformando ele em um código de máquina devidamente otimizado. Inicialmente esse relatório conterá apenas a parte da análise léxica, mas a medida que o período for passando, outros componentes do compilador serão adicionados. O capítulo 2 será referente ao analisador léxico, já o terceiro abordará o sintático, o quarto o semântico e por fim o quinto abordando o desenvolvimento do código alvo que o transforma no código de máquina otimizado.

2. Analisador Léxico

2.1. Uma breve descrição do que foi solicitado

Na primeira etapa do trabalho nos foi solicitado inicialmente escolher uma Linguagem de programação simplificada, sendo esse a linguagem C - por [Louden 2004], definir um subconjunto de regras de produção e que através das expressões regulares geradas pudéssemos gerar um DFA mínimo sem que houvesse qualquer automatização gerada por qualquer tipo de ferramenta. Além disso, tinhamos que gerar o analisador léxico juntamente com um Scanner para todos os Tokens da BNF escolhida. Foi decidido que o scanner seria capaz de ler todas as palavras reservadas e especiais da linguagem proposta por [Louden 2004].

2.2. DFA minimizado e Matriz de Transição

O Autômato Finito Determinístico minimizado que aceita todas as palavras marcadas e caracteres especiais da linguagem C- é demonstrado abaixo.

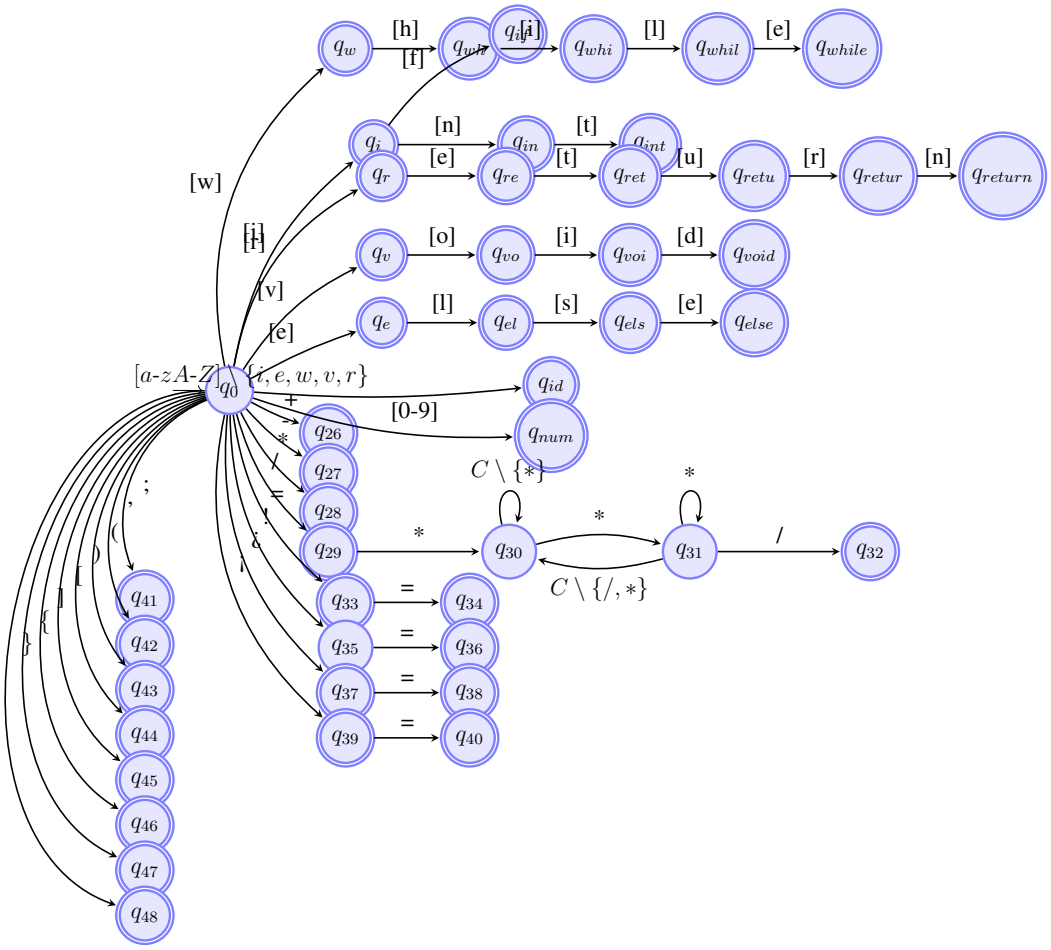


Figura 2. DFA minimizado final unificado com palavras-chave, operadores e símbolos especiais.

Tabela 1. Matriz de Transição do AFD (simplificada por classes de equivalência de caracteres).

Estado	L_{outra}	e	i	r	v	w	d	f	h	l	n	o	s	t	u	[0-9]	=	/	*	!/>/<	Outros
q_0	q_{ID}	q_e	q_i	q_r	q_v	q_w	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{num}	q_{ASSIGN}	$q_/$	q_*	q_{op}	q_{simb}
q_i	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{if}	q_{ID}	q_{ID}	q_{in}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{if}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{in}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{int}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{int}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_w	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{wh}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{wh}	q_{ID}	q_{ID}	q_{whi}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{whi}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{whil}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{whil}	q_{ID}	q_{while}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{while}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_e	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{el}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{el}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{els}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{els}	q_{ID}	q_{else}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{else}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_v	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{vo}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{vo}	q_{ID}	q_{ID}	q_{voi}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{voi}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{void}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{void}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_r	q_{ID}	q_{re}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{re}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ret}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{ret}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{retu}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{retu}	q_{ID}	q_{ID}	q_{ID}	q_{retur}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{retur}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{return}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{return}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{ID}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
q_{num}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{num}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
$q_/$	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	$q_{/*}$	q_{erro}	q_{erro}
$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/**}$	$q_{/*}$	$q_{/*}$
$q_{/**}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{/*}$	$q_{COMMENT}$	$q_{/**}$	$q_{/*}$	$q_{/*}$
q_{ASSIGN}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{EQUAL}	q_{erro}	q_{erro}	q_{erro}	q_{erro}
$q!$	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	$q_{NOTEQUAL}$	q_{erro}	q_{erro}	q_{erro}	q_{erro}
$q>$	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	q_{erro}	$q_{MAIORIGUAL}$	q_{erro}	q_{erro}	q_{erro}	q_{erro}

A Tabela 1 apresenta a matriz de transição do DFA. Para fins de legibilidade e síntese da representação, os caracteres de entrada foram agrupados nas seguintes classes de equivalência:
L_{outras}: Conjunto de letras maiúsculas e minúsculas $[A\text{-}Z, a\text{-}z]$ que não iniciam nem compõem caminhos de palavras reservadas (ex: $a, b, c, g, j, k, m, p, q, x, y, z$).
[0-9]: Todos os dígitos numéricos de 0 a 9.
$!/ > / <$: Operadores relacionais simples que podem compor operadores duplos com $=$ (gerando $q_{NOTEQUAL}$, $q_{MAIORIGUAL}$, etc.).
Outros: Delimitadores e operadores de caractere único $(+, -, ,, ;, (,), [,], \{, \})$ cujas transições a partir de q_0 levam diretamente ao seu estado final individual e geram erro em outros estados.

2.3. O Scanner

2.4. FLEX gerado da BNF

Uma das atividades solicitadas foi a geração do analisador léxico com FLEX, contendo todas as regras da BNF. O propósito principal para a geração do scanner em FLEX é para conferir se as saídas geradas pelo nosso scanner estão corretas. Ele está dividido em 3 partes. Definição, regras e Código. A primeira coisa feita foi a geração de um buffer com capacidade de 4096 bytes, que serve de limite para a escrita dos tokens, a variável linha atual que é um simples contador, muito útil para dar relatórios, indicando em qual linha ocorreu os erros léxicos. Além disso todas as regras envolvendo as palavras reservadas, identificadores, caracteres especiais e números estão indicados da linha 13 até a 68 de acordo com as suas regras. Uma das grandes dificuldades no uso do FLEX pra gerar a BNF foi se acostumar com a sintaxe dos comandos e das regras em si inicialmente. Após isso a maior dificuldade foi na geração do token de comentário, sendo necessário um conjunto de diversas regras que relacionam-se entre si para ser gerado. Uma dificuldade secundária também foi o costume do uso do uso dos comandos pelo terminal, sendo que inicialmente a criação do FLEX foi feita em um compilador online do FLEX chamado JDoodle para depois ser escrito no nosso código.

```
1  %{
2  #include <stdio.h>
3  #include <string.h>
4
5  int linha_atual = 1;
6
7
8  char string_comentario[4096];
9  %}
10
11 %x COMENTARIO
12
13 %%
14 "else"          { printf("(\"%s\", ELSE)\n", yytext); }
15 "if"            { printf("(\"%s\", IF)\n", yytext); }
16 "int"           { printf("(\"%s\", INT)\n", yytext); }
17 "return"        { printf("(\"%s\", RETURN)\n", yytext); }
18 "void"          { printf("(\"%s\", VOID)\n", yytext); }
19 "while"         { printf("(\"%s\", WHILE)\n", yytext); }
20
```

```
"+"             { printf("(\"%s\", SOMA)\n", yytext); }
"-"             { printf("(\"%s\", SUB)\n", yytext); }
"*"             { printf("(\"%s\", MULT)\n", yytext); }
"/"             { printf("(\"%s\", DIV)\n", yytext); }

"<="            { printf("(\"%s\", LEQ)\n", yytext); }
"<"             { printf("(\"%s\", LT)\n", yytext); }
">="            { printf("(\"%s\", GEQ)\n", yytext); }
">"             { printf("(\"%s\", GT)\n", yytext); }
"=="            { printf("(\"%s\", EQ)\n", yytext); }
"!="            { printf("(\"%s\", NEQ)\n", yytext); }
"="             { printf("(\"%s\", ASSIGN)\n", yytext); }

";"             { printf("(\"%s\", SEMI)\n", yytext); }
","             { printf("(\"%s\", COMMA)\n", yytext); }
"("             { printf("(\"%s\", LPAREN)\n", yytext); }
")"             { printf("(\"%s\", RPAREN)\n", yytext); }
"["             { printf("(\"%s\", LBRACK)\n", yytext); }
"]"             { printf("(\"%s\", RBRACK)\n", yytext); }
"{"             { printf("(\"%s\", LBRACE)\n", yytext); }
"}"             { printf("(\"%s\", RBRACE)\n", yytext); }

[0-9]+          { printf("(\"%s\", NUM)\n", yytext); }
[a-zA-Z]+       { printf("(\"%s\", ID)\n", yytext); }

"/*"            {
                    /* Inicializa o buffer com o inicio do comentario */
                    strcpy(string_comentario, "/*");
                    BEGIN(COMENTARIO);
                }
<COMENTARIO>"*/" {
                    /* Finaliza o buffer, imprime o token e volta ao normal */
                    strcat(string_comentario, "*/");
                    printf("(\"%s\", COMENTARIO)\n", string_comentario);
                    BEGIN(INITIAL);
                }
<COMENTARIO>\n   {
                    linha_atual++;
                    strcat(string_comentario, yytext);
                }
<COMENTARIO>.    {
                    strcat(string_comentario, yytext);
                }

\n              { linha_atual++; }
[ \t\r]+        { /* ignora espacos vazios */ }
.               { printf("ERRO LEXICO (Linha %d): Caractere inesperado '%s'\n", linha_atual, yytext); }
%%

int yywrap() { return 1; }
int main() { yylex(); return 0; }
```

Para averiguar o analisador léxico foram utilizadas 4 instâncias corretas e 4 incorretas. Cada uma com o seu devido grau de complexidade, pois já que todas as palavras reservadas e especiais da linguagem C - foram trazidas, foi considerado por bem, exibir todas elas nos testes. Os testes válidos foram chamados de valido1,valido2,...,valido4 e os inválidos de invalido1, invalido2,...,invalido4.

2.5. Resultados obtidos pelo nosso analisador comparado com o scanner gerado pelo FLEX

Os programas para testes podem ser encontrados no Appendix A. Testes e inválidos estão presentes. Já no anexo B encontra-se os resultados obtidos pelo scanner gerado pelo Flex e o C contendo o nosso Scanner.

2.6. Conclusão

A análise Léxica é apenas o primeiro passo para o desenvolvimento do compilador. Conseguimos gerar os tokens para todas as palavras reservadas e caracteres especiais da linguagem C- do [Louden 2004] ao ler a expressão regular representada pelas linhas de código de um programa em C- qualquer. O fato de todas os tipos de tokens terem sido feitos corretamente é um grande passo para a produção de um analisador sintático capaz de gerar todas as regras da BNF no futuro.

Referências

Louden, K. C. (2004). *Compiladores: Princípios e Práticas*. Cengage Learning.

3. Anexos do experimento Envolvendo o Analisador Léxico.

A. Código Fonte dos Programas Validos e Inválidos

```
/* programa fatorial em C- */
int fatorial(int n) {
    if (n <= 1)
        return 1;
    else
        return n * fatorial(n - 1);
}

void main(void) {
    int x;
    x = input();
    output(fatorial(x));
}
```

Figura 3. Programa com resultado válido 1.

```
/* soma os elementos de um vetor
   comentario com mais de uma linha */
int soma(int v[], int n) {
    int i;
    int s;
    s = 0;
    i = 0;
    while (i < n) {
        s = s + v[i];
        i = i + 1;
    }
    return s;
}

void main(void) {
    int a[5];
    int total;
    total = soma(a, 5);
    output(total);
}
```

Figura 4. Programa com resultado válido 2.

```c
int mdc(int u, int v) {
    if (v == 0) return u;
    else return mdc(v, u - u / v * v);
}
```

Figura 5. Programa com resultado válido 3.

```c
int soma(int a, int b) {
    int resultado;
    resultado = a + b;
    while (resultado > 0) {
        resultado = resultado - 1;
    }
    return resultado;
}
```

Figura 6. Programa com resultado válido 4.

```c
int main(void) {
    int x;
    x = 10;
    if (x >= 5) {
        x = x @ 2;
    }
    y# = x & 1;
    return 0;
}
```

Figura 7. Programa com resultado Invalido 1.

```
.
,
&
"
#
/
```

Figura 8. Programa com resultado Invalido 2.

```
int preco;
preco = 45.90; /* ERRO: Caractere inesperado '.' */
```

Figura 9. Programa com resultado Invalido 3.

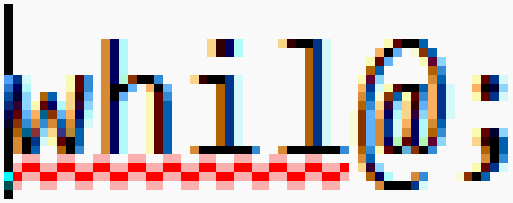


Figura 10. Programa com resultado Invalido 4.

B. Dados dos Testes Realizados do Scanner flex

```
Família@Reobote26 UCRT64 /c/Users/Família/Desktop/TrabalhoDeCompiladores/src
$ ./analisador.exe
/* programa fatorial em C- */
("/* programa fatorial em C- */", COMENTARIO)
int fatorial(int n) {
("int", INT)
("fatorial", ID)
("(", LPAREN)
("int", INT)
("n", ID)
(")", RPAREN)
("{", LBRACE)
    if (n <= 1)
("if", IF)
("(", LPAREN)
("n", ID)
("<=", LEQ)
("1", NUM)
(")", RPAREN)
        return 1;
("return", RETURN)
("1", NUM)
(";", SEMI)
    else
("else", ELSE)
        return n * fatorial(n - 1);
("return", RETURN)
("n", ID)
("*", MULT)
("fatorial", ID)
("(", LPAREN)
("n", ID)
("-", SUB)
("1", NUM)
(")", RPAREN)
(";", SEMI)
}
("}", RBRACE)

void main(void) {
("void", VOID)
("main", ID)
("(", LPAREN)
("void", VOID)
(")", RPAREN)
("{", LBRACE)
    int x;
("int", INT)
("x", ID)
(";", SEMI)
    x = input();
("x", ID)
("=", ASSIGN)
("input", ID)
("(", LPAREN)
(")", RPAREN)
(";", SEMI)
    output(fatorial(x));
("output", ID)
("(", LPAREN)
("fatorial", ID)
("(", LPAREN)
("x", ID)
(")", RPAREN)
(")", RPAREN)
(";", SEMI)
}
("}", RBRACE)
```

Figura 11. Programa com resultado válido 1.

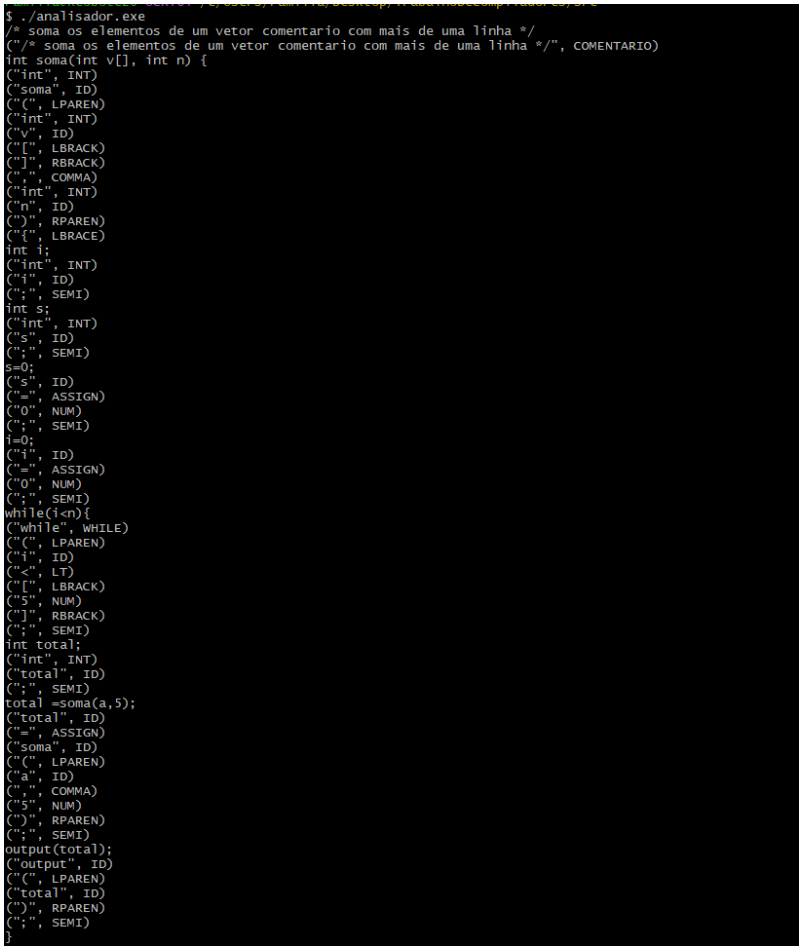


Figura 12. Programa com resultado válido 2.

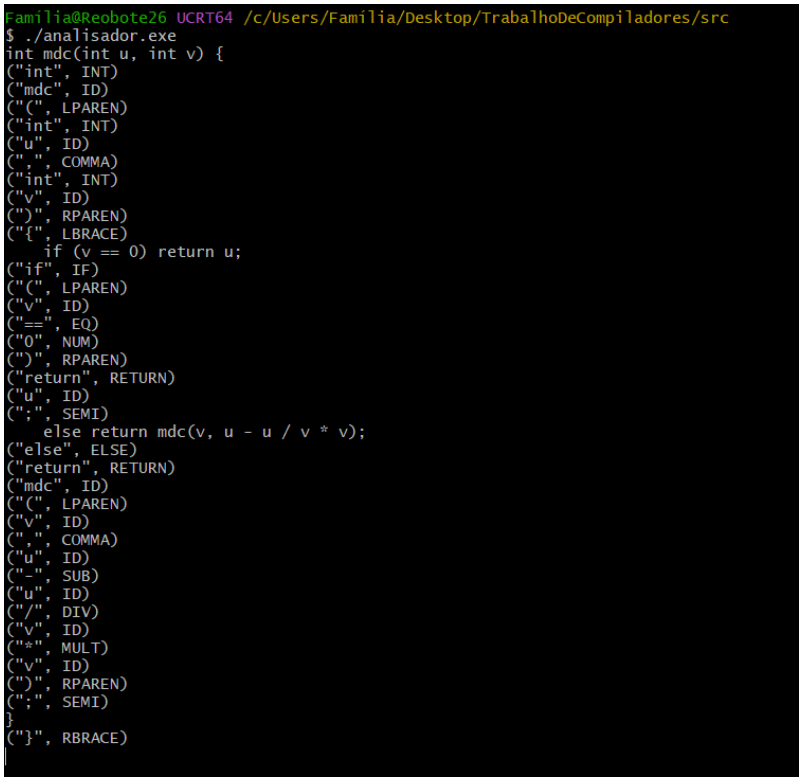


Figura 13. Programa com resultado válido 3.

```
Família@Reobote26 UCRT64 ~
$ cd /c/Users/Família/Desktop/TrabalhoDeCompiladores/src

Família@Reobote26 UCRT64 /c/Users/Família/Desktop/TrabalhoDeCompiladores/src
$ ./analisador.exe
int soma(int a, int b) {
("int", INT)
("soma", ID)
("(", LPAREN)
("int", INT)
("a", ID)
(",", COMMA)
("int", INT)
("b", ID)
(")", RPAREN)
("{", LBRACE)
    int resultado;
("int", INT)
("resultado", ID)
(";", SEMI)
    resultado = a + b;
("resultado", ID)
("=", ASSIGN)
("a", ID)
("+", SOMA)
("b", ID)
(";", SEMI)
   while (resultado > 0) {
("while", WHILE)
("(", LPAREN)
("resultado", ID)
(">", GT)
("0", NUM)
(")", RPAREN)
("{", LBRACE)
        resultado = resultado - 1;
("resultado", ID)
("=", ASSIGN)
("resultado", ID)
("-", SUB)
("1", NUM)
(";", SEMI)
    }
("}", RBRACE)
    return resultado;
("return", RETURN)
("resultado", ID)
(";", SEMI)
}
("}", RBRACE)
```

Figura 14. Programa com resultado válido 4.

```
Família@Reobote26 UCRT64 /c/Users/Família/Desktop/TrabalhoDeCompiladores/src
$ ./analisador.exe
int main(void) {
("int", INT)
("main", ID)
("(", LPAREN)
("void", VOID)
(")", RPAREN)
("{", LBRACE)
    int x;
("int", INT)
("x", ID)
(";", SEMI)
    x = 10;
("x", ID)
("=", ASSIGN)
("10", NUM)
(";", SEMI)
    if (x >= 5) {
("if", IF)
("(", LPAREN)
("x", ID)
(">=", GEQ)
("5", NUM)
(")", RPAREN)
("{", LBRACE)
        x = x @ 2;
("x", ID)
("=", ASSIGN)
("x", ID)
ERRO LEXICO (Linha 5): Caractere inesperado '@'
("2", NUM)
(";", SEMI)
    }
("}", RBRACE)
    y# = x & 1;
("y", ID)
ERRO LEXICO (Linha 7): Caractere inesperado '#'
("=", ASSIGN)
("x", ID)
ERRO LEXICO (Linha 7): Caractere inesperado '&'
("1", NUM)
(";", SEMI)
    return 0;
("return", RETURN)
("0", NUM)
(";", SEMI)
}
("}", RBRACE)
```

Figura 15. Programa com resultado Invalido 1.

```
Família@Reobote26 UCRT64 ~
$ cd /c/Users/Família/Desktop/TrabalhoDeCompiladores/src

Família@Reobote26 UCRT64 /c/Users/Família/Desktop/TrabalhoDeCompiladores/src
$ ./analisador.exe
.
ERRO LEXICO (Linha 1): Caractere inesperado '.'
,
(",", COMMA)
&
ERRO LEXICO (Linha 3): Caractere inesperado '&'
"
ERRO LEXICO (Linha 4): Caractere inesperado '"'
#
ERRO LEXICO (Linha 5): Caractere inesperado '#'
/
```

Figura 16. Programa com resultado Invalido 2.

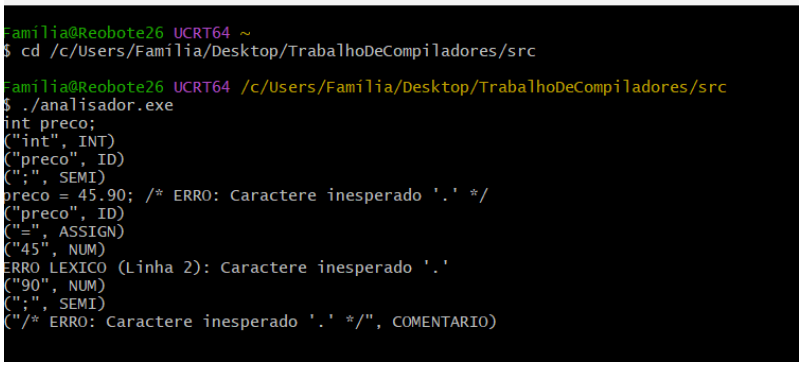


Figura 17. Programa com resultado Invalido 3.



Figura 18. Programa com resultado Invalido 4.

C. Dados dos Testes Realizados do Scanner

```
PS C:\Users\Família\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas> ./programa.exe
Token: /* programa fatorial em C- */ Tipo: TOKEN_COMMENT Linha: 1
Token: int Tipo: TOKEN_INT Linha: 2
Token: fatorial Tipo: TOKEN_IDENTIFIER Linha: 2
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 2
Token: int Tipo: TOKEN_INT Linha: 2
Token: n Tipo: TOKEN_IDENTIFIER Linha: 2
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 2
Token: { Tipo: TOKEN_BRACE_OPEN Linha: 2
Token: if Tipo: TOKEN_IF Linha: 3
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 3
Token: n Tipo: TOKEN_IDENTIFIER Linha: 3
Token: <= Tipo: TOKEN_LESS_EQUAL Linha: 3
Token: 1 Tipo: TOKEN_NUMBER Linha: 3
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 3
Token: return Tipo: TOKEN_RETURN Linha: 4
Token: 1 Tipo: TOKEN_NUMBER Linha: 4
Token: ; Tipo: TOKEN_SEMICOLON Linha: 4
Token: else Tipo: TOKEN_ELSE Linha: 5
Token: return Tipo: TOKEN_RETURN Linha: 6
Token: n Tipo: TOKEN_IDENTIFIER Linha: 6
Token: * Tipo: TOKEN_MULTIPLY Linha: 6
Token: fatorial Tipo: TOKEN_IDENTIFIER Linha: 6
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 6
Token: n Tipo: TOKEN_IDENTIFIER Linha: 6
Token: - Tipo: TOKEN_ERROR Linha: 6
Token: 1 Tipo: TOKEN_NUMBER Linha: 6
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 6
Token: ; Tipo: TOKEN_SEMICOLON Linha: 6
Token: } Tipo: TOKEN_BRACE_CLOSE Linha: 7
Token: void Tipo: TOKEN_VOID Linha: 9
Token: main Tipo: TOKEN_IDENTIFIER Linha: 9
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 9
Token: void Tipo: TOKEN_VOID Linha: 9
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 9
Token: { Tipo: TOKEN_BRACE_OPEN Linha: 9
Token: int Tipo: TOKEN_INT Linha: 10
Token: x Tipo: TOKEN_IDENTIFIER Linha: 10
Token: ; Tipo: TOKEN_SEMICOLON Linha: 10
Token: x Tipo: TOKEN_IDENTIFIER Linha: 11
Token: = Tipo: TOKEN_ASSIGN Linha: 11
Token: input Tipo: TOKEN_IDENTIFIER Linha: 11
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 11
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 11
Token: ; Tipo: TOKEN_SEMICOLON Linha: 11
Token: output Tipo: TOKEN_IDENTIFIER Linha: 12
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 12
Token: fatorial Tipo: TOKEN_IDENTIFIER Linha: 12
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 12
Token: x Tipo: TOKEN_IDENTIFIER Linha: 12
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 12
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 12
Token: ; Tipo: TOKEN_SEMICOLON Linha: 12
Token: } Tipo: TOKEN_BRACE_CLOSE Linha: 13
```

Figura 19. Programa com resultado válido 1 do Scanner.

```
PS C:\Users\Família\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas> ./programa.exe
Token: /* soma os elementos de um vetor
   com Tipo: TOKEN_COMMENT Linha: 1
Token: int Tipo: TOKEN_INT Linha: 3
Token: soma Tipo: TOKEN_IDENTIFIER Linha: 3
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 3
Token: int Tipo: TOKEN_INT Linha: 3
Token: v Tipo: TOKEN_IDENTIFIER Linha: 3
Token: [ Tipo: TOKEN_BRACKET_OPEN Linha: 3
Token: ] Tipo: TOKEN_BRACKET_CLOSE Linha: 3
Token: , Tipo: TOKEN_COMMA Linha: 3
Token: int Tipo: TOKEN_INT Linha: 3
Token: n Tipo: TOKEN_IDENTIFIER Linha: 3
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 3
Token: { Tipo: TOKEN_BRACE_OPEN Linha: 3
Token: int Tipo: TOKEN_INT Linha: 4
Token: i Tipo: TOKEN_IDENTIFIER Linha: 4
Token: ; Tipo: TOKEN_SEMICOLON Linha: 4
Token: int Tipo: TOKEN_INT Linha: 5
Token: s Tipo: TOKEN_IDENTIFIER Linha: 5
Token: ; Tipo: TOKEN_SEMICOLON Linha: 5
Token: s Tipo: TOKEN_IDENTIFIER Linha: 6
Token: = Tipo: TOKEN_ASSIGN Linha: 6
Token: 0 Tipo: TOKEN_NUMBER Linha: 6
Token: ; Tipo: TOKEN_SEMICOLON Linha: 6
Token: i Tipo: TOKEN_IDENTIFIER Linha: 7
Token: = Tipo: TOKEN_ASSIGN Linha: 7
Token: 0 Tipo: TOKEN_NUMBER Linha: 7
Token: ; Tipo: TOKEN_SEMICOLON Linha: 7
Token: while Tipo: TOKEN_WHILE Linha: 8
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 8
Token: i Tipo: TOKEN_IDENTIFIER Linha: 8
Token: < Tipo: TOKEN_LESS Linha: 8
Token: n Tipo: TOKEN_IDENTIFIER Linha: 8
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 8
Token: { Tipo: TOKEN_BRACE_OPEN Linha: 8
Token: s Tipo: TOKEN_IDENTIFIER Linha: 9
Token: = Tipo: TOKEN_ASSIGN Linha: 9
Token: s Tipo: TOKEN_IDENTIFIER Linha: 9
Token: + Tipo: TOKEN_ERROR Linha: 9
Token: v Tipo: TOKEN_IDENTIFIER Linha: 9
Token: [ Tipo: TOKEN_BRACKET_OPEN Linha: 9
Token: i Tipo: TOKEN_IDENTIFIER Linha: 9
Token: ] Tipo: TOKEN_BRACKET_CLOSE Linha: 9
Token: ; Tipo: TOKEN_SEMICOLON Linha: 9
Token: i Tipo: TOKEN_IDENTIFIER Linha: 10
```

Figura 20. Programa com resultado válido 2 do Scanner.

```
PS C:\Users\Família\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas> g++ src/main.cpp src/scann
er.cpp src/Token.cpp -I include -o programa.exe; if ($?) { ./programa.exe }
Token: int Tipo: TOKEN_INT Linha: 1
Token: mdc Tipo: TOKEN_IDENTIFIER Linha: 1
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 1
Token: int Tipo: TOKEN_INT Linha: 1
Token: u Tipo: TOKEN_IDENTIFIER Linha: 1
Token: , Tipo: TOKEN_COMMA Linha: 1
Token: int Tipo: TOKEN_INT Linha: 1
Token: v Tipo: TOKEN_IDENTIFIER Linha: 1
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 1
Token: { Tipo: TOKEN_BRACE_OPEN Linha: 1
Token: if Tipo: TOKEN_IF Linha: 2
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 2
Token: v Tipo: TOKEN_IDENTIFIER Linha: 2
Token: == Tipo: TOKEN_EQUAL_EQUAL Linha: 2
Token: 0 Tipo: TOKEN_NUMBER Linha: 2
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 2
Token: return Tipo: TOKEN_RETURN Linha: 2
Token: u Tipo: TOKEN_IDENTIFIER Linha: 2
Token: ; Tipo: TOKEN_SEMICOLON Linha: 2
Token: else Tipo: TOKEN_ELSE Linha: 3
Token: return Tipo: TOKEN_RETURN Linha: 3
Token: mdc Tipo: TOKEN_IDENTIFIER Linha: 3
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 3
Token: v Tipo: TOKEN_IDENTIFIER Linha: 3
Token: , Tipo: TOKEN_COMMA Linha: 3
Token: u Tipo: TOKEN_IDENTIFIER Linha: 3
Token: - Tipo: TOKEN_ERROR Linha: 3
Token: u Tipo: TOKEN_IDENTIFIER Linha: 3
Token: / Tipo: TOKEN_DIVIDE Linha: 3
Token: v Tipo: TOKEN_IDENTIFIER Linha: 3
Token: * Tipo: TOKEN_MULTIPLY Linha: 3
Token: v Tipo: TOKEN_IDENTIFIER Linha: 3
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 3
Token: ; Tipo: TOKEN_SEMICOLON Linha: 3
Token: } Tipo: TOKEN_BRACE_CLOSE Linha: 4
PS C:\Users\Família\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas>
```

Figura 21. Programa com resultado válido 3 do Scanner.

```
PS C:\Users\Família\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas> ./programa.exe
Token: int Tipo: TOKEN_INT Linha: 1
Token: soma Tipo: TOKEN_IDENTIFIER Linha: 1
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 1
Token: int Tipo: TOKEN_INT Linha: 1
Token: a Tipo: TOKEN_IDENTIFIER Linha: 1
Token: , Tipo: TOKEN_COMMA Linha: 1
Token: int Tipo: TOKEN_INT Linha: 1
Token: b Tipo: TOKEN_IDENTIFIER Linha: 1
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 1
Token: { Tipo: TOKEN_BRACE_OPEN Linha: 1
Token: int Tipo: TOKEN_INT Linha: 2
Token: resultado Tipo: TOKEN_IDENTIFIER Linha: 2
Token: ; Tipo: TOKEN_SEMICOLON Linha: 2
Token: resultado Tipo: TOKEN_IDENTIFIER Linha: 3
Token: = Tipo: TOKEN_ASSIGN Linha: 3
Token: a Tipo: TOKEN_IDENTIFIER Linha: 3
Token: + Tipo: TOKEN_ERROR Linha: 3
Token: b Tipo: TOKEN_IDENTIFIER Linha: 3
Token: ; Tipo: TOKEN_SEMICOLON Linha: 3
Token: while Tipo: TOKEN_WHILE Linha: 4
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 4
Token: resultado Tipo: TOKEN_IDENTIFIER Linha: 4
Token: > Tipo: TOKEN_GREATER Linha: 4
Token: 0 Tipo: TOKEN_NUMBER Linha: 4
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 4
Token: { Tipo: TOKEN_BRACE_OPEN Linha: 4
Token: resultado Tipo: TOKEN_IDENTIFIER Linha: 5
Token: = Tipo: TOKEN_ASSIGN Linha: 5
Token: resultado Tipo: TOKEN_IDENTIFIER Linha: 5
Token: - Tipo: TOKEN_ERROR Linha: 5
Token: 1 Tipo: TOKEN_NUMBER Linha: 5
Token: ; Tipo: TOKEN_SEMICOLON Linha: 5
Token: } Tipo: TOKEN_BRACE_CLOSE Linha: 6
Token: return Tipo: TOKEN_RETURN Linha: 7
Token: resultado Tipo: TOKEN_IDENTIFIER Linha: 7
Token: ; Tipo: TOKEN_SEMICOLON Linha: 7
Token: } Tipo: TOKEN_BRACE_CLOSE Linha: 8
PS C:\Users\Família\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas>
```

Figura 22. Programa com resultado válido 4 do Scanner.

```
PS C:\Users\Família\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas> ./programa.exe
Token: int Tipo: TOKEN_INT Linha: 1
Token: main Tipo: TOKEN_IDENTIFIER Linha: 1
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 1
Token: void Tipo: TOKEN_VOID Linha: 1
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 1
Token: { Tipo: TOKEN_BRACE_OPEN Linha: 1
Token: int Tipo: TOKEN_INT Linha: 2
Token: x Tipo: TOKEN_IDENTIFIER Linha: 2
Token: ; Tipo: TOKEN_SEMICOLON Linha: 2
Token: x Tipo: TOKEN_IDENTIFIER Linha: 3
Token: = Tipo: TOKEN_ASSIGN Linha: 3
Token: 10 Tipo: TOKEN_NUMBER Linha: 3
Token: ; Tipo: TOKEN_SEMICOLON Linha: 3
Token: if Tipo: TOKEN_IF Linha: 4
Token: ( Tipo: TOKEN_PAREN_OPEN Linha: 4
Token: x Tipo: TOKEN_IDENTIFIER Linha: 4
Token: >= Tipo: TOKEN_GREATER_EQUAL Linha: 4
Token: 5 Tipo: TOKEN_NUMBER Linha: 4
Token: ) Tipo: TOKEN_PAREN_CLOSE Linha: 4
Token: { Tipo: TOKEN_BRACE_OPEN Linha: 4
Token: x Tipo: TOKEN_IDENTIFIER Linha: 5
Token: = Tipo: TOKEN_ASSIGN Linha: 5
Token: x Tipo: TOKEN_IDENTIFIER Linha: 5
Token: @ Tipo: TOKEN_ERROR Linha: 5
Token: 2 Tipo: TOKEN_NUMBER Linha: 5
Token: ; Tipo: TOKEN_SEMICOLON Linha: 5
Token: } Tipo: TOKEN_BRACE_CLOSE Linha: 6
Token: y Tipo: TOKEN_IDENTIFIER Linha: 7
Token: # Tipo: TOKEN_ERROR Linha: 7
Token: = Tipo: TOKEN_ASSIGN Linha: 7
Token: x Tipo: TOKEN_IDENTIFIER Linha: 7
Token: & Tipo: TOKEN_ERROR Linha: 7
Token: 1 Tipo: TOKEN_NUMBER Linha: 7
Token: ; Tipo: TOKEN_SEMICOLON Linha: 7
Token: return Tipo: TOKEN_RETURN Linha: 8
Token: 0 Tipo: TOKEN_NUMBER Linha: 8
Token: ; Tipo: TOKEN_SEMICOLON Linha: 8
Token: } Tipo: TOKEN_BRACE_CLOSE Linha: 9
```

Figura 23. Programa com resultado inválido 1 do Scanner.

```
PS C:\Users\Família\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas> ./programa.exe
Token: . Tipo: TOKEN_ERROR Linha: 1
Token: , Tipo: TOKEN_COMMA Linha: 2
Token: & Tipo: TOKEN_ERROR Linha: 3
Token: " Tipo: TOKEN_ERROR Linha: 4
Token: # Tipo: TOKEN_ERROR Linha: 5
Token: / Tipo: TOKEN_DIVIDE Linha: 6
PS C:\Users\Família\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas>
```

Figura 24. Programa com resultado inválido 2 do Scanner.

```
PS C:\Users\Familia\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas> ./programa.exe
Token: int Tipo: TOKEN_INT Linha: 1
Token: preco Tipo: TOKEN_IDENTIFIER Linha: 1
Token: ; Tipo: TOKEN_SEMICOLON Linha: 1
Token: preco Tipo: TOKEN_IDENTIFIER Linha: 2
Token: = Tipo: TOKEN_ASSIGN Linha: 2
Token: 45 Tipo: TOKEN_NUMBER Linha: 2
Token: . Tipo: TOKEN_ERROR Linha: 2
Token: 90 Tipo: TOKEN_NUMBER Linha: 2
Token: ; Tipo: TOKEN_SEMICOLON Linha: 2
Token: /* ERRO: Caractere inesperado '.' */ Tipo: TOKEN_COMMENT Linha: 2
PS C:\Users\Familia\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas>
```

Figura 25. Programa com resultado inválido 3 do Scanner.

```
PS C:\Users\Familia\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas> ./programa.exe
Token: whil Tipo: TOKEN_IDENTIFIER Linha: 1
Token: @ Tipo: TOKEN_ERROR Linha: 1
Token: ; Tipo: TOKEN_SEMICOLON Linha: 1
PS C:\Users\Familia\Downloads\TrabalhoDeCompiladores-Lucas\TrabalhoDeCompiladores-Lucas>
```

Figura 26. Programa com resultado inválido 4 do Scanner.